

Министерство науки и профессионального образования
Забайкальского края
Государственное автономное профессиональное образовательное учреждение
«Забайкальский горный колледж имени М.И.Агошкова»

ОТЧЕТ

По преддипломной практики

ИП Ермакова И.С., пгт.Новокручининский ул.Фабричная, 5В
(Название, Местонахождение предприятия)

Разработка мобильного приложения для автоматизации деятельности персонального фитнес-тренера с функцией удаленного мониторинга прогресса клиента
(Тема Проекта)

Выполнил Ермаков Н.С. ИСиП-22-1
ФИО студента, группа

подпись

Руководитель практики от предприятия

_____ Ермакова И.С. Директор
подпись ФИО, должность

Руководитель практики от колледжа

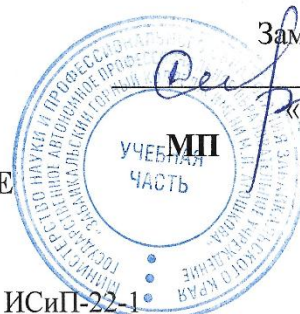
_____ Гладышева А.В.
подпись ФИО, должность

Дата

МП

Чита 2026

ЗАДАНИЕ



на преддипломную практику: 144 часа
с 20.04.2026 по 16.05.2026

студенту (ке) **Ермакову Никите Сергеевичу** группы **ИСИП-22-1**
специальность 09.02.07. Информационные системы и программирование

Тема дипломного проекта: **Разработка мобильного приложения для автоматизации деятельности персонального фитнес-тренера с функцией удаленного мониторинга прогресса клиента**

За период прохождения практики на предприятии **Магазин Бегемотик** необходимо ознакомиться с деятельностью предприятия, приобрести практический опыт по видам профессиональной деятельности:

Вид практики	Вид профессиональной деятельности
преддипломная	Разработка модулей программного обеспечения для компьютерных систем Осуществление интеграции программных модулей Сопровождение и обслуживание программного обеспечения компьютерных систем Разработка и администрирование баз данных

а также собрать материалы для составления отчета по практике; составить отчет и заверить его подписью руководителя от предприятия и печатью предприятия, вести дневник практики.

Задание на практику подшивается в отчет по производственной практике после титульного листа.

Разделы отчёта по практике.

Введение

1. Общие сведения о предприятии (история создания, развитие и структура предприятия, программное обеспечение предприятия, базы данных, используемые на предприятии).
2. Техника безопасности на предприятии.
3. Примерное содержание практики.
4. Разработка программного продукта.
 - 4.1. Техническое задание.
 - 4.2. Проектирование программного продукта (описание предметной области, модульная структура программы, диаграммы вариантов использования, ERD- диаграммы, вайрфрейм).
 - 4.3. Обоснование выбора средств создания ПП.
 - 4.4. Программирование.
 - 4.5. Тестирование и отладка программного продукта.
 - 4.6. Документирование.

Заключение.

Список литературы.

Приложение 1 Код программы

Приложение 2 Направление на практику

Приложение 3 Аттестационный лист

Приложение 4 Характеристика

Приложение 5 Дневник производственной практики

Приложение 6 Диск с программным продуктом

Примечание.

1. К отчету прилагается графический материал (схемы, иллюстрации по всем разделам, фотографии, модульная структура программы).
2. В заключении студент составляет самоанализ практики, поводит итоги, делает анализ выполнения программы практики и приобретенных профессиональных умений и навыков.
3. К отчету прилагается программный продукт, разработанный за время прохождения практики, который должен содержать базу данных не менее чем из 3-х таблиц.

Руководитель практики

Гладышева А.В.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ОБЩИЕ СВЕДЕНИЯ О ПРЕДПРИЯТИИ	6
2. ТЕХНИКА БЕЗОПАСНОСТИ НА ПРЕДПРИЯТИИ.....	7
3. ПРИМЕРНОЕ СОДЕРЖАНИЕ ПРАКТИКИ	8
4. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА.....	10
4.1 Техническое задание	10
4.2 Проектирование программного продукта.....	11
4.3 Обоснование выбора средств создания ПП.	18
4.4 Программирование.....	20
4.5 Тестирование и отладка программного продукта	23
4.6 Документирование программного продукта	26
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ 1. Код программы.....	Ошибка! Закладка не определена.
ПРИЛОЖЕНИЕ 2. Направление на практику	Ошибка! Закладка не определена.
ПРИЛОЖЕНИЕ 3. Аттестационный лист ...	Ошибка! Закладка не определена.
ПРИЛОЖЕНИЕ 4. Характеристика	Ошибка! Закладка не определена.
ПРИЛОЖЕНИЕ 5. Дневник производственной практики	Ошибка! Закладка не определена.
ПРИЛОЖЕНИЕ 6. Диск с программным продуктом.....	Ошибка! Закладка не определена.

ВВЕДЕНИЕ

Преддипломная практика является завершающим и очень важным этапом обучения по специальности «Информационные системы и программирование». Она позволяет не только закрепить теоретические знания, но и применить их на практике, приобрести реальный опыт разработки программного обеспечения, сбора материалов для дипломного проекта и оформления технической документации. Практика проходила в магазине детских товаров «Бегемотик» под руководством директора Ермаковой Ирины Станиславовны. В рамках практики была поставлена задача разработать мобильное приложение «SportTrackler» — Android-приложение, которое автоматизирует взаимодействие фитнес-тренера с клиентами: ведение клиентской базы, общение в режиме реального времени через встроенный чат, отслеживание прогресса клиентов, ведение дневника питания, замеров тела и фотофиксации результатов. Цель практики — создание полноценного программного продукта для автоматизации деятельности персонального фитнес-тренера с функцией удалённого мониторинга прогресса клиента, готового к внедрению в реальную деятельность. Для достижения поставленной цели были поставлены следующие задачи:

- провести анализ предметной области и изучить существующее программное обеспечение предприятия;
- сформулировать техническое задание на разработку приложения;
- выбрать подходящие технологии и инструменты разработки;
- спроектировать пользовательский интерфейс, архитектуру системы и базы данных;
- реализовать все модули приложения на языке Kotlin с использованием архитектурного паттерна MVVM;
- разработать серверная часть (REST API) на платформе ASP.NET Core с базой данных SQL Server и развёрнуть на удалённом сервере;
- выполнить тестирование и отладка приложения;
- подготовить техническую документацию и отчёт по практике.

За время практики был пройден полный цикл разработки — от идеи до готового продукта, что позволило сформировать целостное представление о работе мобильного разработчика и получить ценные профессиональные навыки.

1. ОБЩИЕ СВЕДЕНИЯ О ПРЕДПРИЯТИИ

Магазин детских товаров «Бегемотик» был основан в 2024 году в посёлке Новокручининский. Идея открытия данного магазина принадлежала Ермакову Никите Сергеевичу и Ермаковой Ирине Станиславовне.

С начальным капиталом было выкуплено помещение, в котором в дальнейшем был сделан капитальный ремонт, было сварено отопление, установлены камеры видеонаблюдения, также была установлена сигнализация и завезены стеллажи. Первая партия товаров была небольшой, были закуплены игрушки и немного детской одежды, весь акцент был сделан на качество товара, что в дальнейшем положительно сказалось на впечатлении покупателей.

Так постепенно магазин развивался, и с каждым месяцем товаров становилось всё больше, ассортимент расширялся исходя из просьб покупателей: было закуплено детское питание, памперсы, канцелярия, электроника и многое другое.

На сегодняшний день «Бегемотик» представляет собой современный магазин детских товаров с разнообразным ассортиментом и высокими стандартами обслуживания. Структура магазина включает в себя следующие ключевые элементы:

- Отделы товаров: игрушки, канцелярия, детская одежда, детское питание, детская химия, электроника, книжки.

- Маркетинг и реклама: активная работа с социальными сетями для привлечения клиентов и информирования о новинках и акциях, разнос визиток в другие магазины.

- Обслуживание клиентов: умение рекомендовать товар по запросам клиентов, приём заказов для закупки по просьбам клиентов.

2.ТЕХНИКА БЕЗОПАСНОСТИ НА ПРЕДПРИЯТИИ

Обеспечение безопасности в магазине детских товаров — это важная задача, которая включает в себя как физическую безопасность, так и защиту данных. В магазине «Бегемотик» действует ряд инструкций по охране труда и технике безопасности, с которыми был проведён инструктаж при поступлении на практику.

1) Физическая безопасность. Эта категория включает в себя установку камер видеонаблюдения, которые помогают предотвратить кражи и обеспечить контроль за происходящим в магазине; ограничение доступа в служебные помещения и на склад.

2) Безопасность сотрудников. Регулярное ознакомление сотрудников с правилами техники безопасности, включая действия в экстренных ситуациях. Разработка и размещение планов эвакуации на случай пожара или других чрезвычайных ситуаций.

3) Безопасность товаров. Регулярные проверки запасов и соблюдение норм хранения товаров.

4) Безопасность данных. Обновление программного обеспечения и систем безопасности для защиты от угроз, использование надёжных паролей, замена терминала каждый год при его аренде.

5) Общие правила поведения на рабочем месте. Соблюдать порядок и чистоту на рабочем месте, не оставлять без присмотра включённое электрооборудование, при обнаружении неисправности оборудования немедленно сообщать руководителю.

6) Охрана труда при работе с персональным компьютером. Перед началом работы проверить исправность компьютера, монитора, клавиатуры и периферийных устройств, не допускать попадания жидкости на оборудование. Запрещается самостоятельно вскрывать и ремонтировать оборудование.

3. ПРИМЕРНОЕ СОДЕРЖАНИЕ ПРАКТИКИ

Прохождение практики было организовано в соответствии с календарным планом и охватывало все этапы жизненного цикла разработки программного обеспечения.

Первая неделя, с 20 по 26 апреля, была посвящена организационным вопросам и анализу предметной области. Было изучена внутренняя документация предприятия, обсуждены с руководителем требования к будущей системе и сформулированы цели и задачи проекта. Был проведён анализ существующих аналогов мобильных приложений для фитнеса и выявлены ключевые потребности: необходимость оперативного общения тренера с клиентами, ведения дневников питания, замеров и прогресса, а также фотофиксации результатов.

Вторая неделя, с 27 апреля по 3 мая, включала проектирование программного продукта. На этом этапе разрабатывались макеты пользовательского интерфейса, создавались диаграммы вариантов использования (Use Case), диаграммы последовательности и деятельности для основных процессов, таких как авторизация, отправка сообщения и добавление записи в дневник питания. Также была спроектирована структура базы данных, определены сущности «Пользователи», «Сообщения», «Замеры», «Записи питания», «Фотографии» и связи между ними.

Третья неделя, с 4 по 10 мая, стала периодом активного программирования. Была настроена среда разработки Android Studio, создан проект на языке Kotlin с архитектурой MVVM, реализовано подключение к серверному REST API через библиотеку Retrofit, настроено подключение к SignalR для работы чата в режиме реального времени. Разработаны основные экраны приложения: авторизация, регистрация, панель тренера, панель клиента, чат, дневник питания, замеры, фотографии прогресса.

Четвёртая, заключительная неделя, с 11 по 16 мая, была направлена на тестирование, отладку и документирование. Проводилось тестирование всех функций приложения, исправлялись выявленные ошибки, такие как некорректное

отображение сообщений в чате при слабом соединении. После успешного тестирования было составлено руководство пользователя и инструкция по установке, оформлен отчёт по практике.

Весь процесс работы фиксировался в дневнике практики, который ежедневно подписывался руководителем от предприятия.

4. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА

4.1 Техническое задание

Техническое задание на разработку мобильного приложения «SportTracker» было составлено в период прохождения преддипломной практики. Требования к системе разделены на функциональные и нефункциональные.

Функциональные требования включают подсистему аутентификации и ограничения прав доступа. Пользователь должен вводить логин и пароль, после чего система проверяет их соответствие записям на сервере, а также определяет роль пользователя. В зависимости от роли пользователю предоставляется соответствующий набор функций. Тренер имеет доступ к списку своих клиентов, может просматривать их профили, вести переписку, отслеживать замеры, питание и фотографии каждого клиента. Клиент может вести дневник питания с поиском продуктов и указанием калорийности, добавлять замеры тела (вес, обхват груди, талии, бёдер и другие параметры), загружать фотографии прогресса, общаться с тренером через встроенный чат.

Модуль чата должен обеспечивать обмен текстовыми сообщениями в режиме реального времени с использованием технологии SignalR. Также должна поддерживаться отправка голосовых сообщений с функцией записи, паузы и отмены, отправка видеосообщений, изображений и файлов. Интерфейс чата должен отображать статус соединения и время отправки сообщений.

Модуль дневника питания должен позволять добавлять записи о приёмах пищи с указанием продукта, количества, калорийности и содержания белков, жиров и углеводов. Должна быть реализована функция поиска продуктов по базе данных.

Модуль замеров должен позволять добавлять записи замеров тела с датой и числовыми значениями параметров, а также отображать историю изменений.

Модуль фотографий прогресса должен позволять загружать фотографии с камеры или галереи и просматривать историю фотографий.

Нефункциональные требования. Приложение разрабатывается на языке

Kotlin для операционной системы Android версии 8.0 и выше. Архитектура приложения — MVVM (Model-View-ViewModel). Интерфейс должен быть на русском языке, использовать единую тёмную цветовую гамму. Время отклика на типовые операции не должно превышать 2 секунд при наличии стабильного соединения. Серверная часть взаимодействует с приложением через REST API, для чата используется протокол SignalR. Должна быть реализована обработка ошибок сети с выводом понятных пользователю сообщений. Состав документации — руководство пользователя и инструкция по установке в формате PDF. Техническое задание утверждено руководителем практики и является неизменным в ходе разработки.

4.2 Проектирование программного продукта

Проектирование программного продукта являлось ключевым этапом, на котором выполнен переход от общего понимания требований к конкретной архитектуре будущего приложения. В данном разделе приводится описание предметной области, модульная структура, диаграмма вариантов использования, ERD-диаграмма базы данных и макеты интерфейса.

Предметная область охватывает деятельность фитнес-тренера, оказывающего услуги персональных тренировок. Основными участниками являются тренер и клиенты. Центральные бизнес-процессы включают регистрацию и авторизацию пользователей, привязку клиента к тренеру, общение через чат, ведение клиентом дневника питания и замеров, загрузку фотографий прогресса, а также просмотр тренером данных своих клиентов. Каждый пользователь имеет профиль с именем, фамилией, адресом электронной почты, номером телефона и аватаром. Роль определяет набор доступных функций. Сообщения в чате содержат текст, тип вложения, ссылку на медиафайл и дату отправки. Запись питания описывается названием продукта, количеством, калорийностью, содержанием белков, жиров и углеводов, а также датой приёма пищи. Замер описывается датой и набором числовых параметров тела. Фотография прогресса содержит ссылку на изображение и дату загрузки. Разграничение прав доступа

реализовано так, что клиент не может просматривать данные других клиентов, а тренер не может редактировать данные клиента — только просматривать.

На основе анализа предметной области была спроектирована многоуровневая модульная архитектура. Все модули логически разделены на три уровня: уровень представления, уровень бизнес-логики и уровень доступа к данным. Такое разделение обеспечивает независимую разработку и облегчает тестирование.

На уровне представления выделены следующие модули: модуль авторизации (LoginFragment) — обеспечивает ввод логина и пароля и взаимодействует с AuthViewModel; модуль регистрации (RegisterFragment) — реализует создание новой учётной записи; главный модуль навигации (MainActivity) — содержит нижнюю панель навигации, которая динамически настраивается в зависимости от роли вошедшего пользователя; модуль тренера (ClientDashboardFragment) — отображает список клиентов тренера; модуль чата (ChatFragment) — обеспечивает переписку в режиме реального времени с поддержкой голосовых и видеосообщений; модуль дневника питания (FoodDiaryFragment) — позволяет добавлять и просматривать записи питания; модуль замеров (MeasurementsFragment) — отображает историю замеров тела; модуль фотографий прогресса (PhotosFragment) — показывает фотографии в хронологическом порядке.

Уровень бизнес-логики представлен ViewModel-классами, соответствующими каждому модулю представления, а также моделями данных, отражающими структуру ответов сервера. Уровень доступа к данным реализован через ApiService (интерфейс Retrofit для REST-запросов), RetrofitClient (настройка клиента с добавлением токена авторизации), TokenStorage (хранение токена и идентификатора пользователя).

Для наглядного описания функциональных требований была построена диаграмма прецедентов. На диаграмме изображены акторы: «Тренер» и «Клиент».

Для тренера выделены следующие варианты использования: «Войти в систему», «Просмотреть список клиентов», «Открыть чат с клиентом», «Просмотреть замеры клиента», «Просмотреть дневник питания клиента», «Просмотреть фотографии клиента». Для клиента: «Войти в систему», «Открыть чат с тренером», «Добавить запись питания», «Добавить замер», «Загрузить фотографию прогресса». Также присутствует общий вариант использования «Аутентификация», который включается во все сценарии входа.

На рисунке 1 размещена диаграмма прецедентов.

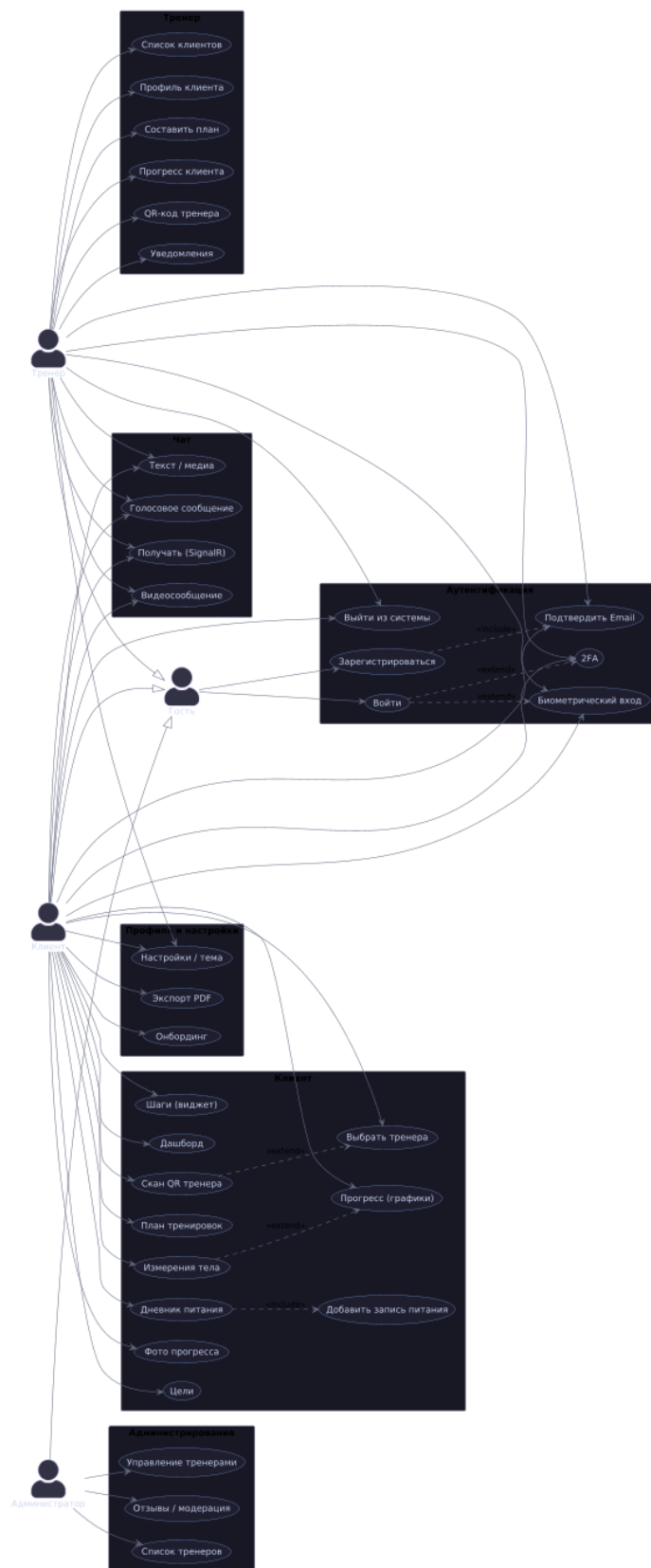


Рисунок 1 – Диаграмма прецедентов.

ERD-диаграмма (модель «сущность-связь»). Для проектирования базы данных была создана логическая модель. Основные сущности: «Пользователи» —

атрибуты: идентификатор, логин, пароль, имя, фамилия, email, телефон, роль, путь к аватару, дата создания. «Роли» — идентификатор, название роли. «Сообщения» — идентификатор, идентификатор отправителя (внешний ключ), идентификатор получателя (внешний ключ), текст, тип вложения, URL вложения, дата отправки, флаг прочтения. «Записи питания» — идентификатор, идентификатор пользователя (внешний ключ), название продукта, количество в граммах, калорийность, белки, жиры, углеводы, дата. «Замеры» — идентификатор, идентификатор пользователя (внешний ключ), дата, вес, обхват груди, обхват талии, обхват бёдер, другие параметры. «Фотографии прогресса» — идентификатор, идентификатор пользователя (внешний ключ), URL фотографии, дата загрузки. «Связь тренер-клиент» — идентификатор тренера (внешний ключ), идентификатор клиента (внешний ключ). Связи: один-ко-многим между «Пользователями» и «Сообщениями», между «Пользователями» и «Записями питания», между «Пользователями» и «Замерами», между «Пользователями» и «Фотографиями».

На рисунке 2 размещена ER-модель базы данных.

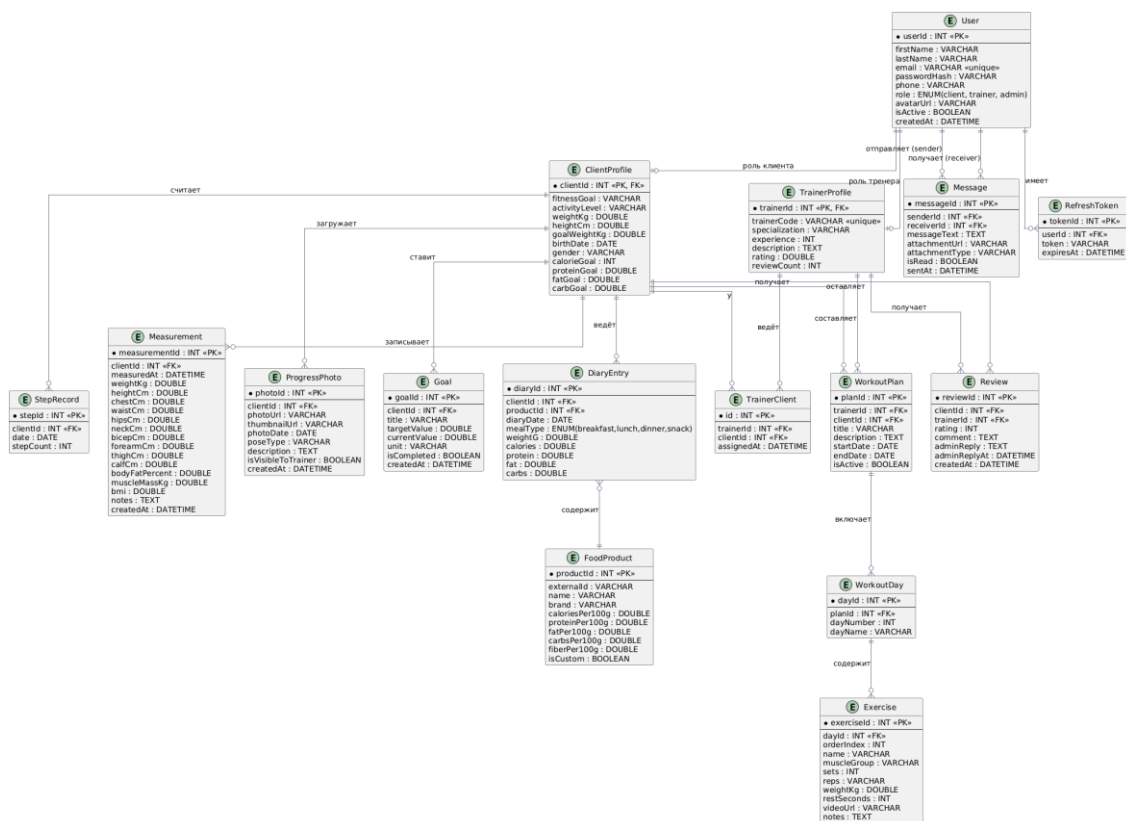


Рисунок 2 – ER модель базы данных.

Макеты интерфейса. Прежде чем приступить к кодированию, были созданы макеты всех основных экранов. Макеты позволили согласовать внешний вид и логику навигации с руководителем практики. Макет экрана авторизации содержит поля «Логин», «Пароль», кнопку «Войти» и ссылку на экран регистрации. Макет главного экрана тренера включает список клиентов с именем, аватаром и кнопкой перехода в чат. Макет экрана чата содержит список сообщений, поле ввода текста, кнопки прикрепления файла, записи голосового сообщения и видеосообщения. Макет дневника питания отображает список записей за день с суммарной калорийностью и кнопкой добавления новой записи. Макет замеров содержит форму ввода параметров и историю предыдущих замеров. Все макеты выполнены в едином тёмном стиле с удобными отступами, что обеспечивает интуитивную понятность для пользователей.

На рисунке 3 размещён макет экрана авторизации

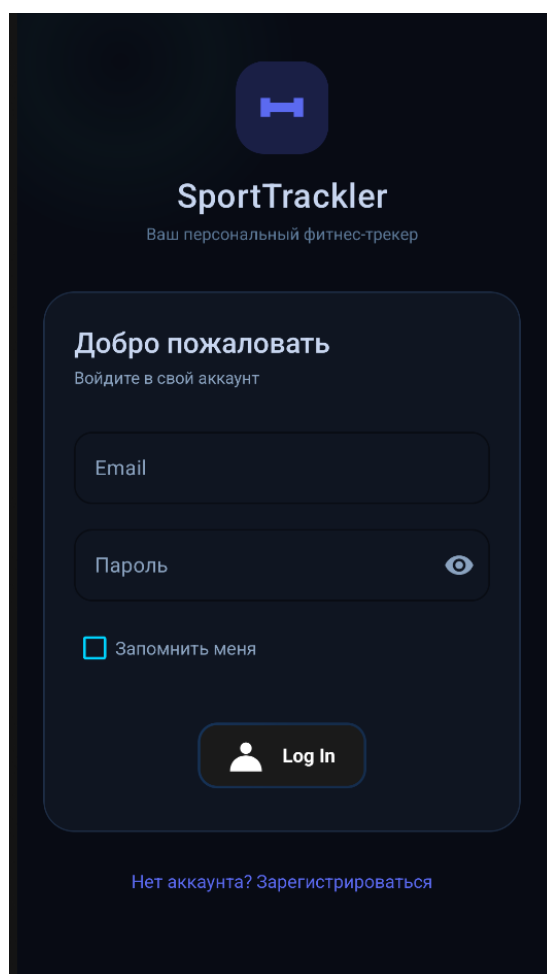


Рисунок 3 – Макет окна авторизации.

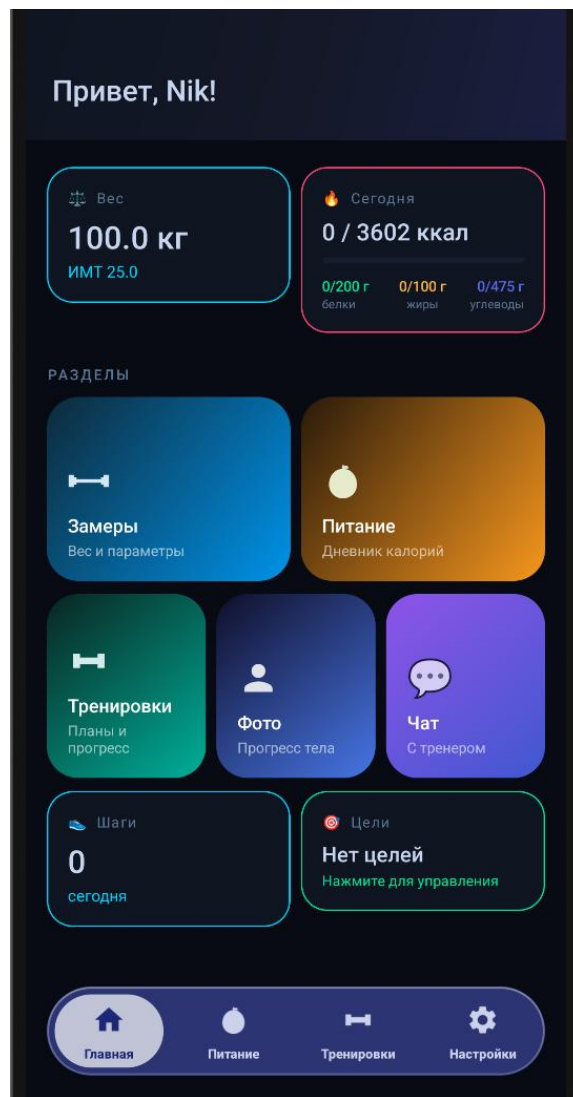


Рисунок 4 – Макет главного окна клиента.

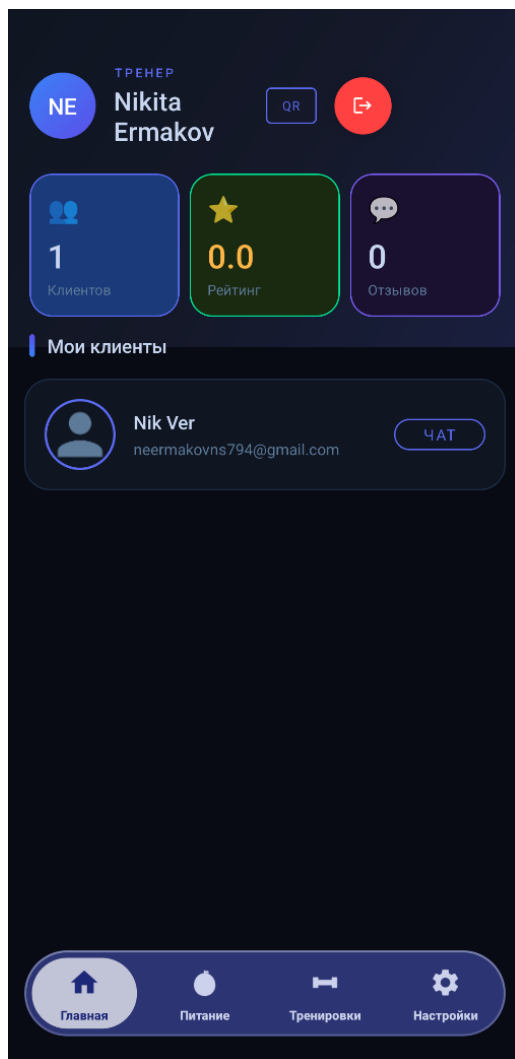


Рисунок 5 – Макет окна создания заказа.

4.3 Обоснование выбора средств создания ПП.

Выбор технологий и инструментов для разработки мобильного приложения «SportTracker» осуществлялся с учётом требований к функциональности, надёжности, производительности, удобству последующего сопровождения и развёртывания. Были проанализированы возможные альтернативы по каждому компоненту: языку программирования, платформе разработки, архитектурному паттерну, способу взаимодействия с сервером и среде разработки. В результате был выбран язык Kotlin, платформа Android SDK, архитектурный паттерн MVVM, библиотека Retrofit для REST-запросов, библиотека SignalR для чата в реальном времени, среда разработки Android Studio.

Основным языком реализации выбран Kotlin. Этот язык является современ-

ным, лаконичным и безопасным языком программирования, официально поддерживаемым Google для разработки Android-приложений. Kotlin поддерживает такие важные концепции, как корутины (coroutines) для асинхронного программирования, расширения функций, null-безопасность, лямбда-выражения. Альтернативой мог служить Java, однако Kotlin значительно сокращает объём кода, снижает вероятность ошибок типа NullPointerException и предоставляет более современный синтаксис. Также рассматривался Flutter (язык Dart) как кроссплатформенное решение, однако он требует дополнительного изучения и имеет ограничения при работе с нативными Android-компонентами. Поэтому выбор Kotlin является наиболее обоснованным.

Для организации кода был выбран архитектурный паттерн MVVM (Model-View-ViewModel). Данный паттерн официально рекомендован Google в рамках Android Architecture Components. MVVM обеспечивает чёткое разделение ответственности между слоями: View отвечает только за отображение данных, ViewModel содержит бизнес-логику и взаимодействует с репозиторием, Model представляет данные. Применение LiveData и ViewModelScope упрощает управление жизненным циклом компонентов. Альтернативой является MVP (Model-View-Presenter), однако он менее гибок и требует большего объёма шаблонного кода. Поэтому выбор MVVM является оптимальным.

Для взаимодействия с серверным REST API была выбрана библиотека Retrofit. Retrofit является наиболее популярной и широко используемой библиотекой для HTTP-запросов в Android-разработке. Она предоставляет декларативный способ описания API-эндпоинтов через интерфейсы, автоматически конвертирует JSON-ответы в объекты Kotlin через Gson, поддерживает корутины. Альтернативой является OkHttp, однако он требует значительно большего объёма кода для выполнения тех же операций. Выбор Retrofit является стандартом индустрии для Android.

Для реализации чата в режиме реального времени была выбрана технология SignalR. SignalR обеспечивает двунаправленную связь между клиентом и сер-

вером через WebSocket, автоматически переключаясь на Long Polling при недоступности WebSocket. Сервер приложения уже использует SignalR Hub, что делает использование клиентской библиотеки SignalR для Android естественным выбором. Альтернативой является Firebase Realtime Database, однако она требует использования инфраструктуры Google и изменения серверной части.

Для написания кода использовалась среда разработки Android Studio. Эта среда является официальной средой разработки для Android, предоставляет мощный редактор кода с автодополнением, встроенный эмулятор устройств, визуальный редактор макетов, профилировщик памяти и производительности, а также интеграцию с системой контроля версий Git. Выбор Android Studio является безальтернативным для профессиональной разработки под Android.

Таким образом, выбор каждого компонента был обусловлен требованиями к надёжности, производительности, простоте разработки и дальнейшего сопровождения. Применённый стек технологий является стандартным для современной Android-разработки и позволил успешно реализовать все функциональные требования, предъявленные к приложению «SportTracker».

4.4 Программирование

Программирование является центральным этапом разработки, на котором все проектные решения, зафиксированные в техническом задании, диаграммах и макетах, были воплощены в работающий программный код. Разработка велась в среде Android Studio. Был создан новый проект типа «Empty Activity» на языке Kotlin с минимальной версией Android API 26 (Android 8.0). Для организации кода в соответствии с архитектурой MVVM в структуре проекта были созданы пакеты: data (модели, сетевой слой, локальное хранилище), ui (фрагменты и ViewModel для каждого экрана), di (вспомогательные классы). Такой подход обеспечил разделение ответственности: модели представляют данные, ViewModel содержит бизнес-логику, фрагменты отвечают только за отображение и ввод данных.

В пакете data.model определены классы данных (data class), соответствующие

структурам JSON-ответов сервера. Класс UserProfile включает поля userId, firstName, lastName, email, phone, avatarUrl, roleId. Класс MessageDto содержит id, senderId, receiverId, content, attachmentType, mediaUrl, sentAt. Класс FoodEntry описывает запись питания с полями id, userId, productName, grams, calories, protein, fat, carbs, date. Класс Measurement содержит id, userId, date и числовые параметры замеров. Класс PhotoProgress включает id, userId, photoUrl, takenAt. Все классы используют аннотации Gson для корректной сериализации JSON.

В пакете data.network определён интерфейс ApiService, описывающий все эндпоинты REST API. Метод login() выполняет POST-запрос к /api/auth/login и возвращает токен авторизации. Метод getMessages() выполняет GET-запрос к /api/messages/{contactId} и возвращает список сообщений. Метод sendMessage() выполняет POST-запрос к /api/messages и сохраняет новое сообщение. Метод getMyClients() возвращает список клиентов тренера. Метод uploadChatMedia() выполняет загрузку файла на сервер через Multipart-запрос. Метод getFoodEntries() и addFoodEntry() обеспечивают работу с дневником питания. Методы getMeasurements() и addMeasurement() обеспечивают работу с замерами. Методы getPhotos() и uploadPhoto() обеспечивают работу с фотографиями прогресса. Все методы используют аннотации Retrofit и возвращают объекты Response<T> для корректной обработки ошибок.

Класс RetrofitClient выполняет настройку HTTP-клиента. При каждом запросе в заголовок Authorization автоматически добавляется Bearer-токен, извлекаемый из TokenStorage. Базовый URL задаётся через BuildConfig.BASE_URL, что позволяет легко переключаться между окружениями.

Класс TokenStorage реализован с использованием EncryptedSharedPreferences для безопасного хранения токена доступа и идентификатора пользователя на устройстве.

ViewModel-классы реализованы с использованием viewModelScope для запуска корутин. AuthViewModel обрабатывает логику входа и регистрации, хранит состояние аутентификации через LiveData. ChatViewModel управляет под-

ключением к SignalR Hub, хранит список сообщений, реализует методы startVoiceRecording(), stopVoiceRecordingAndSend(), pauseVoiceRecording(), resumeVoiceRecording(), cancelVoiceRecording() для работы с голосовыми сообщениями через MediaRecorder. FoodDiaryViewModel управляет загрузкой и добавлением записей питания. MeasurementsViewModel управляет историей замеров. PhotosViewModel обеспечивает загрузку и отображение фотографий прогресса. ClientDashboardViewModel загружает список клиентов тренера.

ChatFragment реализует сложный интерфейс чата: поддерживает переключение между режимами ввода текста, записи голосового сообщения и отправки. При записи голосового сообщения реализовано управление жестами — свайп влево для отмены, свайп вверх для фиксации записи. Фиксированный режим записи предоставляет кнопки паузы, удаления и отправки. Для круговых видеосообщений реализован VideoNoteBottomSheet.

На рисунке 6 размещена диаграмма классов



. Рисунок 6 –Диаграмма классов.

Все разработанные компоненты системы строго соответствуют требованиям технического задания и обеспечивают удобное разделение функционала между тренером и клиентом в соответствии с их ролями.

4.5 Тестирование и отладка программного продукта

Тестирование и отладка являются неотъемлемыми этапами жизненного

цикла разработки программного обеспечения. Целью данного этапа являлось выявление и устранение ошибок в работе мобильного приложения «SportTracker», а также проверка соответствия реализованного функционала требованиям технического задания. Тестирование проводилось в ручном режиме по заранее разработанным сценариям, охватывающим все ключевые функции. В процессе тестирования также проверялась корректность работы интерфейса и обработка некорректных действий пользователя.

Для систематизации процесса тестирования были составлены детальные сценарии. Каждый сценарий включает идентификатор тестового примера, приоритет, описание действий, тестовые данные, ожидаемый и фактический результат. Примеры разработанных сценариев представлены в таблицах 1, 2, 3..

Таблица 1 – Сценарий тестирования

Поле	Описание
Название проекта	SportTracker
Рабочая версия	1.0
Имя тестирующего	Ермаков Н.С
Тестовый пример	ТС_AUTH_001
Приоритет тестирования	Высокий
Заголовок теста	Проверка успешной аутентификации пользователей с различными ролями
Этапы теста	1. Запустить приложение. 2. В поле «Логин» ввести логин тренера. 3. В поле «Пароль» ввести пароль. 4. Нажать кнопку «Войти». 5. Повторить для учётной записи клиента.
Тестовые данные	Логин тренера: trainer@test.ru, пароль: 123456. Логин клиента: client@test.ru, пароль: 123456.
Ожидаемый результат	Тренеру отображается экран со списком клиентов. Клиенту отображается экран с личным кабинетом.
Фактический результат	Совпадает с ожидаемым.
Статус	Зачёт
Предварительное условие	Приложение запущено, сервер доступен.
Постусловие	Пользователь авторизован в системе.

Таблица 2 – Сценарий тестирования

Поле	Описание
Название проекта	SportTracker
Рабочая версия	1.0
Имя тестирующего	Ермаков Н.С
Тестовый пример	ТС_ЧАТ_001
Приоритет тестирования	Высокий

Продолжение таблицы 2

Заголовок теста	Проверка отправки и получения текстового сообщения в чате
Этапы теста	1. Авторизоваться под учётной записью тренера. 2. Открыть чат с клиентом. 3. Ввести текст сообщения «Тестовое сообщение». 4. Нажать кнопку отправки. 5. Проверить отображение сообщения в чате
Тестовые данные	Текст: «Тестовое сообщение»
Ожидаемый результат	Сообщение отображается в чате со временем отправки. На устройстве клиента сообщение появляется в режиме реального времени.
Фактический результат	Совпадает с ожидаемым.
Статус	Зачёт
Предварительное условие	Оба пользователя авторизованы, соединение с сервером установлено.
Постусловие	Сообщение сохранено в базе данных.

Таблица 3 – Сценарий тестирования

Поле	Описание
Название проекта	SportTracker
Рабочая версия	1.0
Имя тестирующего	Ермаков Н.С
Тестовый пример	ТС_FOOD_001
Приоритет тестирования	Средний
Заголовок теста	Проверка добавления записи в дневник питания
Этапы теста	1. Авторизоваться под учётной записью клиента. 2. Перейти в раздел «Дневник питания». 3. Нажать кнопку добавления записи. 4. Ввести название продукта «Гречка», вес 150 г, калорийность 162 ккал. 5. Сохранить запись.
Тестовые данные	Продукт: «Гречка», вес: 150 г, калорийность: 162 ккал
Ожидаемый результат	Запись отображается в списке дневника питания с указанными данными. Суммарная калорийность за день обновляется.
Фактический результат	Совпадает с ожидаемым.
Статус	Зачёт
Предварительное условие	Пользователь авторизован как клиент.
Постусловие	Запись сохранена в базе данных.

Таким образом, проведённое тестирование подтвердило работоспособность всех основных функций системы, корректность обработки ошибок и соответствие продукта техническому заданию. Программный продукт готов к опытной эксплуатации.

4.6 Документирование программного продукта

Заключительным этапом разработки мобильного приложения «SportTracker» стала подготовка комплекта сопроводительной документации. Документирование является важной частью процесса создания программного обеспечения, поскольку качественно составленные документы облегчают процесс внедрения системы, её освоения пользователями, а также последующего сопровождения и модификации. В ходе работы были разработаны два основных вида документации — пользовательская документация (руководство пользователя) и техническая документация (инструкция по установке и настройке). Все документы были подготовлены в электронном виде в формате PDF, что обеспечивает их читаемость на любом устройстве без необходимости установки дополнительного программного обеспечения.

Пользовательская документация представлена в виде руководства пользователя. Данный документ предназначен для тренеров и клиентов, которые будут непосредственно работать с приложением. Руководство написано простым и понятным языком, избегая излишней технической детализации. Структура руководства включает следующие разделы. В разделе «Введение» кратко описывается назначение приложения и его основные возможности. В разделе «Требования к устройству» перечислены минимальные технические требования: версия Android не ниже 8.0, наличие подключения к интернету, наличие камеры и микрофона. Раздел «Установка» содержит пошаговую инструкцию по установке APK-файла приложения на устройство. Раздел «Авторизация» описывает процесс входа в систему и восстановления пароля. Далее следуют разделы для каждой роли: для тренера описан процесс просмотра списка клиентов и работы с чатом, для клиента — работа с дневником питания, замерами и фотографиями прогресса.

Вся документация подготовлена в соответствии с требованиями ГОСТ 2.105-79 к текстовым документам. Документы включены в состав поставки на электронном носителе вместе с APK-файлом приложения. Документирование вы-

полнено в полном объёме, что позволяет любому пользователю без технических знаний установить приложение и приступить к работе.

ЗАКЛЮЧЕНИЕ

Преддипломная практика, проведённая на базе магазина детских товаров «Бегемотик», стала важным этапом в формировании профессиональных компетенций по специальности «Информационные системы и программирование». Основная цель практики, заключавшаяся в разработке мобильного приложения «SportTracker», была полностью достигнута. За время практики был пройден полный цикл создания программного продукта от анализа предметной области и составления технического задания до проектирования, программирования, тестирования и документирования.

В ходе работы над проектом были выполнены все поставленные задачи. Проведён детальный анализ предметной области, выявлены ключевые потребности пользователей. На основе этого анализа было составлено техническое задание, в котором определены функциональные и нефункциональные требования к системе. На этапе проектирования разработаны макеты пользовательского интерфейса, построены диаграмма прецедентов, диаграмма деятельности и ERD-диаграмма базы данных. Все проектные решения были согласованы с руководителем практики.

В ходе программирования на языке Kotlin с использованием архитектурного паттерна MVVM была реализована многоуровневая архитектура. Разработаны классы для доступа к данным через REST API, ViewModel-классы для бизнес-логики, а также экраны приложения: авторизация, список клиентов, чат с поддержкой голосовых и видеосообщений, дневник питания, замеры и фотографии прогресса. Для обмена сообщениями в режиме реального времени использована технология SignalR.

В процессе тестирования были разработаны сценарии, охватывающие все ключевые функции, и проведена отладка. Выявленные ошибки были устранены.

Таким образом, программа преддипломной практики выполнена в полном объёме. В результате было создано готовое к опытной эксплуатации мобильное

приложение «SportTracker», которое может быть внедрено в деятельность фитнес-тренера для автоматизации взаимодействия с клиентами. Полученный опыт будет использован при выполнении дипломного проекта и в дальнейшей профессиональной деятельности.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 2.105-79. Общие требования к текстовым документам. - М.: Издательство стандартов, 1979. - 36 с.
2. Боровков, В.А. Разработка программных модулей на языке С#: учебное пособие для студентов среднего профессионального образования / В.А. Боровков. - М.: КУДИЦ-Образ, 2022. - 415 с.
3. Зыков Н.В. ЕДИНЫЕ ТРЕБОВАНИЯ К ОФОРМЛЕНИЮ КУРСОВОГО И ДИПЛОМНОГО ПРОЕКТА (РАБОТЫ): методические указания для студентов очного и заочного обучения всех специальностей — 4-е изд., испр. и доп. / Н.В. Зыков. — Чита: ЗабГК, 2020. — 53 с.
4. Троелсен, Э. Язык программирования С# 7 и платформы .NET и .NET Core / Э. Троелсен, Ф. Джепикс. - М.: Вильямс, 2019. - 1328 с.
5. Рихтер, Д. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# / Д. Рихтер. - СПб.: Питер, 2021. - 896 с.
6. Шилдт, Г. C# 4.0: полное руководство / Г. Шилдт. - М.: Вильямс, 2018. - 1056 с.
7. Макконнелл, С. Совершенный код. Мастер-класс / С. Макконнелл. - М.: Русская редакция, 2020. - 896 с.
8. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. - М.: Вильямс, 2019. - 544 с.
9. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. - СПб.: Питер, 2020. - 368 с.
10. Бондаренко, С.А. Проектирование и разработка баз данных в Microsoft SQL Server: учебное пособие / С.А. Бондаренко. - М.: БХВ-Петербург, 2021. - 384 с.
11. Виейра, Р. Программирование баз данных Microsoft SQL Server 2019. Базовый курс / Р. Виейра. - М.: Диалектика, 2020. - 816 с.
12. Бен-Ган, И. Microsoft SQL Server 2019. Основы T-SQL / И. Бен-Ган. - СПб.: Питер, 2021. - 448 с.

					<i>ПДП Преддипломная практика</i>	Лист
						30
Изм.	Лист	№ докум	Подпись	Дата		

13. Куликов, С.С. Тестирование программного обеспечения. Базовый курс / С.С. Куликов. - М.: Издательские решения, 2020. - 302 с.

14. Орлов, С.А. Технологии разработки программного обеспечения: учебник для вузов / С.А. Орлов, Б.Я. Цилькер. - СПб.: Питер, 2019. - 608 с.

15. Лаврищева, Е.М. Программная инженерия. Парадигмы, технологии и CASE-средства: учебник для вузов / Е.М. Лаврищева. - М.: Юрайт, 2020. - 280 с.

16. Visual Paradigm Online. Создание UML-диаграмм [Электронный ресурс]. — Режим доступа: <https://online.visualparadigm.com/app/diagrams/> (дата обращения: 29.05.2026).

17. Microsoft Docs. Документация по .NET и C# [Электронный ресурс]. — Режим доступа: <https://docs.microsoft.com/ru-ru/dotnet/> (дата обращения: 14.05.2026)

Приложение 1 Код приложения

```
package com.fitnesstrainer.app.ui.client

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.view.animation.DecelerateInterpolator
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.lifecycleScope
import androidx.navigation.fragment.findNavController
import androidx.recyclerview.widget.LinearLayoutManager
import com.fitnesstrainer.app.App
import com.fitnesstrainer.app.R
import com.fitnesstrainer.app.databinding.DialogLeaveReviewBinding
import com.fitnesstrainer.app.databinding.FragmentClientDashboardBinding
import com.bumptech.glide.Glide
```



```
import com.bumptech.glide.load.resource.bitmap.CircleCrop

import com.fitnesstrainer.app.ui.news.NewsAdapter

import com.google.android.material.snackbar.Snackbar

import kotlinx.coroutines.launch

import java.time.LocalDate

import java.time.format.DateTimeFormatter

import java.util.Locale

class ClientDashboardFragment : Fragment() {

    private var _binding: FragmentClientDashboardBinding? = null

    private val binding get() = _binding!!

    private val viewModel: ClientDashboardViewModel by viewModels()

    override fun onCreateView(

        inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?

    ): View {

        _binding = FragmentClientDashboardBinding.inflate(inflater, container,

false)

        return binding.root

    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
```

```
super.onViewCreated(view, savedInstanceState)
```

```
setupDate()
```

```
setupClicks()
```

```
setupObservers()
```

```
viewModel.load()
```

```
viewModel.loadTrainer()
```

```
viewModel.loadAvatar()
```

```
loadStepsAndGoals()
```

```
binding.swipeRefresh.setColorSchemeResources(R.color.accent_cyan,  
R.color.accent_indigo)
```

```
binding.swipeRefresh.setOnRefreshListener {
```

```
    viewModel.load()
```

```
    viewModel.loadTrainer()
```

```
    viewModel.loadAvatar()
```

```
    loadStepsAndGoals()
```

```
}
```

```
val newsAdapter = NewsAdapter { item ->
```

```
    findNavController().navigate(
```

```

ClientDashboardFragmentDirections.actionClientDashboardToNews-
WebView(
    url = item.url, title = item.title
)
)
}

binding.rvNews.layoutManager = LinearLayoutManager(requireContext(),
LinearLayoutManager.HORIZONTAL, false)

binding.rvNews.adapter = newsAdapter

viewModel.news.observe(viewLifecycleOwner) { newsAdapter.submitList(it)
}

viewModel.loadNews()

animateEntrance()
}

private fun setupDate() {
    val today = LocalDate.now()

    val formatter = DateTimeFormatter.ofPattern("EEEE, d MMMM", Lo-
cale("ru"))

    binding.tvDate.text = today.format(formatter).replaceFirstChar { it.upper-
case() }
}

```

```

private fun setupClicks() {

    binding.cardMeasurements.setOnClickListener {

        findNavController().navigate(

            ClientDashboardFragmentDirections.actionClientDashboardToMeasure-
ments(-1)

        )

    }

    binding.cardFood.setOnClickListener {

        findNavController().navigate(R.id.action_clientDashboard_to_food)

    }

    binding.cardPhotos.setOnClickListener {

        findNavController().navigate(

            ClientDashboardFragmentDirections.actionClientDashboardToPhotos(-
1)

        )

    }

    binding.cardWorkout.setOnClickListener {

        findNavController().navigate(

            ClientDashboardFragmentDirections.actionClientDash-
boardToWorkout(-1)

        )

    }

    binding.btnScanQr.setOnClickListener {

```

```

        findNavController().navigate(R.id.action_clientDashboard_to_qrScan)
    }

    binding.cardGoals.setOnClickListener {

        findNavController().navigate(R.id.action_clientDashboard_to_goals)

    }

}

```

```

private fun setupObservers() {

    viewModel.state.observe(viewLifecycleOwner) { state ->

        when (state) {

            is DashboardState.Loading -> {

                binding.shimmerLayout.visibility = View.VISIBLE

                binding.shimmerLayout.startShimmer()

                binding.contentLayout.visibility = View.GONE

            }

            is DashboardState.Success -> {

                binding.shimmerLayout.stopShimmer()

                binding.shimmerLayout.visibility = View.GONE

                binding.contentLayout.visibility = View.VISIBLE

                binding.swipeRefresh.isRefreshing = false

                val d = state.data

                val firstName = d.firstName

```

```

binding.tvWelcome.text = "Привет, $firstName!"

val initials = firstName.trim().split(" ")

    .filter { it.isNotEmpty() }.take(2)

    .joinToString("") { it.first().uppercaseChar().toString() }

binding.tvAvatarInitials.text = initials.ifEmpty { firstName.take(1).up-
percase() }

d.latest?.let { m ->

    binding.tvWeight.text = m.weightKg?.let { "%.1f кг".format(it) } ?:
"__"

    binding.tvBmi.text    = m.bmi?.let { "ИМТ %.1f".format(it) } ?: ""
}

d.todaySummary?.let { s ->

    binding.tvCaloriesToday.text =

        "%.0f / %d ккал".format(s.totalCalories, s.calorieGoal ?: 2000)

    val goal = (s.calorieGoal ?: 2000).toFloat()

    binding.progressCalories.max    = 100

    binding.progressCalories.progress = ((s.totalCalories / goal) *
100).toInt().coerceIn(0, 100)

    binding.tvProteinDash.text = if (s.proteinGoal != null)

        "%.0f/%.0f".format(s.totalProtein, s.proteinGoal)

```

```

else "%.0fr".format(s.totalProtein)

binding.tvFatDash.text = if (s.fatGoal != null)

    "%.0f/%.0fr".format(s.totalFat, s.fatGoal)

else "%.0fr".format(s.totalFat)

binding.tvCarbsDash.text = if (s.carbGoal != null)

    "%.0f/%.0fr".format(s.totalCarbs, s.carbGoal)

else "%.0fr".format(s.totalCarbs)

}

}

is DashboardState.Error -> {

    binding.shimmerLayout.stopShimmer()

    binding.shimmerLayout.visibility = View.GONE

    binding.contentLayout.visibility = View.VISIBLE

    binding.swipeRefresh.isRefreshing = false

    Snackbar.make(binding.root, state.message, Snackbar.LENGTH_IN-
DEFINITE)

        .setAction("Повторить") { viewModel.load() }

        .show()

    }

}

}

viewModel.avatarUrl.observe(viewLifecycleOwner) { url ->

```

```

if (!url.isNullOrBlank()) {

    binding.ivAvatar.visibility = View.VISIBLE

    binding.tvAvatarInitials.visibility = View.GONE

    Glide.with(this)

        .load(url)

        .transform(CircleCrop())

        .into(binding.ivAvatar)

} else {

    binding.ivAvatar.visibility = View.GONE

    binding.tvAvatarInitials.visibility = View.VISIBLE

}

}

viewModel.myTrainer.observe(viewLifecycleOwner) { trainer ->

    if (trainer != null) {

        binding.cardReview.visibility = View.VISIBLE

        binding.btnLeaveReview.setOnClickListener {

            showReviewDialog(trainer.userId, "${trainer.firstName}

${trainer.lastName}")

        }

    } else {

        binding.cardReview.visibility = View.GONE

    }

}

```



```

    }
}

private fun animateEntrance() {
    val interp = DecelerateInterpolator(1.5f)
    val views = listOf(
        binding.header,
        binding.sectionNews,
        binding.labelSections,
        binding.navGrid
    )
    views.forEachIndexed { i, v ->
        v.alpha = 0f
        v.translationY = 32f
        v.animate()
            .alpha(1f)
            .translationY(0f)
            .setDuration(450)
            .setStartDelay(i * 70L)
            .setInterpolator(interp)
            .start()
    }
}

```

```

    }           else -> backup.collectionId ?: -1L
    }

    repository.saveNote(
        noteId = null, type = type,
        title = backup.title.orEmpty(),
        content = backup.content.orEmpty(),
        tagNames = backup.tags.orEmpty(),
        imageUrl = backup.imageUrl.orEmpty(),
        excerpt = backup.excerpt.orEmpty(),
        siteName = backup.siteName.orEmpty(),
        createdAt = backup.createdAt.takeIf { it > 0 },
        isFavorite = backup.isFavorite ?: false,
        collectionId = collId
    )

    count++

}

onDone(count)

}

}

private fun groupByDate(list: List<NoteWithTags>): List<ListItem> {

    val today    = dayStart(0)

```

```

val yesterday = dayStart(-1)

val weekAgo = dayStart(-7)

val result = mutableListOf<ListItem>()

var lastLabel: String? = null

list.forEach { nwt ->

    val label = when {

        nwt.note.createdAt >= today    -> "Сегодня"

        nwt.note.createdAt >= yesterday -> "Вчера"

        nwt.note.createdAt >= weekAgo  -> "На этой неделе"

        else                          -> "Ранее"

    }

    if (label != lastLabel) {

        result.add(ListItem.Header(label))

        lastLabel = label

    }

    result.add(ListItem.Note(nwt))

}

return result

}

private fun dayStart(offsetDays: Int): Long {

    val cal = Calendar.getInstance().apply {

```

```
private fun loadStepsAndGoals() {  
    lifecycleScope.launch {  
        val db    = App.instance.database  
        val uid    = App.instance.tokenStorage.getUserId()  
        val today  = LocalDate.now().toString()  
  
        val steps = db.stepDao().get(uid, today)?.steps ?: 0  
        _binding?.tvSteps?.text = steps.toString()  
  
        val goals = db.goalDao().getAll(uid)  
        val done  = goals.count { goal ->  
            val latest = db.measurementDao().getAll(uid).firstOrNull()  
            val cur = latest?.let { m -> when (goal.type) {  
                "WEIGHT"    -> m.weightKg  
                "BODY_FAT"  -> m.bodyFatPercent  
                "CHEST"     -> m.chestCm  
                "WAIST"     -> m.waistCm  
                "HIPS"      -> m.hipsCm  
                "BICEP"     -> m.bicepCm
```

МИНИСТЕРСТВО НАУКИ И ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ ЗАБАЙКАЛЬСКОГО КРАЯ
ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
«ЗАБАЙКАЛЬСКИЙ ГОРНЫЙ КОЛЛЕДЖ ИМЕНИ М.И. АГОШКОВА»

672039, РФ
Забайкальский край
г. Чита, ул. Баргузинская, 41

факс, тел. 8 (3022) 41-69-74
8 (3022) 41-70-38
E-mail: zabgc@mail.ru

Направление на практику

Студент (ка) Ермаков Никита Сергеевич 4 курс, группа ИСиП-22-1
специальности 09.02.07 Информационные системы и программирование

В соответствии с Договором о практической подготовке обучающихся
направляется для прохождения производственной практики, в

Магазин Бегемотик, Забайкальский край, пгт. Новокручининский, ул. Фабричная, 5в
организация, адрес

Срок прохождения практики с 20.04.2026 г. по 16.05.2026 г. 4 недели

Основание: приказ № _____ от «17» апреля 2026 г.

Заведующий отделением ИТ и Э Лапина С.Н.

Выбыл из ГАПОУ «ЗабГК им. М.И. Агошкова», г. Чита «18» апреля 2026 г.

Директор

Н.В. Зыков

М.П.

Зав. отделением ИТ и Э

С.Н. Лапина

Отметки о прибытии на практику и выбытии с практики

Прибыл в ИП Бегемотик
(наименование предприятия)

Убыл ИП Бегемотик из
(наименование предприятия)

«20» 04 2026г.

«16» 05 2026г.

директор
(должность ответственного от предприятия)

директор
(должность ответственного от предприятия)

Ермаков Н.С.
(подпись) | Ермаков Н.С.
(И.О. Фамилия)

Ермаков Н.С.
(подпись) | Ермаков Н.С.
(И.О. Фамилия)

М.П.

М.П.

Цель практики: закрепление и систематизация знаний, полученных при изучении общепрофессиональных дисциплин и профессиональных модулей, формирование общих и профессиональных компетенций по данной специальности, приобретение практического опыта.

АТТЕСТАЦИОННЫЙ ЛИСТ
по итогам преддипломной практики по профилю специальности

ФИО Ермаков Никита Сергеевич,
(ФИО обучающегося (-ейся))

обучающийся (-аяся) по специальности 09.02.07 Информационные системы и программирование,

успешно прошел (ла) преддипломную практику по профессиональным модулям:

ПМ.01. Разработка модулей программного обеспечения для компьютерных систем

ПМ.02. Осуществление интеграции программных модулей

ПМ.03. Сопровождение и обслуживание программного обеспечения компьютерных систем

ПМ 04. Разработка и администрирование баз данных.

В объеме 144 часов, с 20.04.2026 по 16.05.2026

в организации ИП Ермакова И.С., пгт Новокручининский ул.Фабричная, 5В
(наименование организации, юридический адрес)

Виды и качество выполнения работ

Виды и объем работ, выполненные обучающимся во время практики (указывается практический опыт согласно ФГОС СПО)	Освоил	Не освоил	Данный вид работ не проводился	Качество выполнения работ в соответствии с технологией и (или) требованиями организации*
Формировать алгоритмы разработки программных модулей в соответствии с техническим заданием.	Освоил			5
Разрабатывать программные модули в соответствии с техническим заданием.	Освоил			5
Выполнять отладку программных модулей с использованием специализированных программных средств	Освоил			5
Выполнять тестирование программных модулей	Освоил			5
Осуществлять рефакторинг и оптимизацию программного кода	Освоил			5
Разрабатывать модули программного обеспечения для мобильных платформ	Освоил			5
Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент.	Освоил			5
Выполнять интеграцию модулей в программное обеспечение.	Освоил			5
Выполнять отладку программного модуля с использованием специализированных программных средств.	Освоил			5
Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.	Освоил			5

Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.	Освоил			5
Осуществлять установку, настройку и обслуживание программного обеспечения компьютерных систем.	Освоил			5
Осуществлять измерения эксплуатационных характеристик программного обеспечения компьютерных систем.	Освоил			5
Выполнять работы по модификации отдельных компонент программного обеспечения в соответствии с потребностями заказчика.	Освоил			5
Обеспечивать защиту программного обеспечения компьютерных систем программными средствами.	Освоил			5
Осуществлять сбор, обработку и анализ информации для проектирования баз данных	Освоил			5
Проектировать базу данных на основе анализа предметной области	Освоил			5
Разрабатывать объекты базы данных в соответствии с результатами анализа предметной области	Освоил			5
Реализовывать базу данных в конкретной системе управления базами данных	Освоил			5
Администрировать базы данных	Освоил			5
Защищать информацию в базе данных с использованием технологии защиты	Освоил			5

*- критерии оценки (степень соответствия с технологией и (или) требованиям организации, в которой проходила практика): 80–100% - «5»; 65–80% - «4»; 50–65% - «3»; менее 50% - «не освоил»

Дата «15» мая 2026г.

Подпись руководителя практики от колледжа

ЕРМАКОВА
ИРИНА
СТАНИСЛАВОВ
НА

Подписано цифровой
подписью: ЕРМАКОВА
ИРИНА
СТАНИСЛАВОВНА
Дата: 2026.05.22
11:41:28 +09'00'

Подпись ответственного лица организации



Производственная характеристика

Практикант

Ермаков Никита Сергеевич

(ФИО)

За время прохождения практики Ермаков Никита Сергеевич зарекомен-
довал себя как ответственный, исполнительный и приятный в общении со-
трудник. Быстро влился в коллектив, всегда приходил на помощь коллегам и
с интересом относился к новым задачам. Добросовестно выполнял поруче-
ния, не боялся задавать вопросы и учиться новому. Проявил себя как дисци-
плинированный, вежливый и инициативный человек, с которым комфортно
работать в команде.

Оценка за практику отлично

Руководитель практики

Ерш
(подпись)

Ермакова И.С.
(ФИО руководителя)

Дата « 15 » мая 2026 г

ЕРМАКОВА
ИРИНА
СТАНИСЛАВОВ
НА

Подписано цифровой
подписью: ЕРМАКОВА
ИРИНА
СТАНИСЛАВОВНА
Дата: 2026.05.22
11:41:28 +09'00'

МИНИСТЕРСТВО НАУКИ И ПРОФЕССИОНАЛЬНО ОБРАЗОВАНИЯ
ЗАБАЙКАЛЬСКОГО КРАЯ
ГАПОУ
ЗАБАЙКАЛЬСКИЙ ГОРНЫЙ КОЛЛЕДЖ ИМЕНИ М.И. АГОШКОВА

ДНЕВНИК ПО ПРЕДДИПЛОМНОЙ ПРАКТИКЕ

Студента группы: ИСиП-22-1 Ермакова Никиты Сергеевича

(группа, фамилия, имя, отчество)

По специальности: 09.02.07 Информационные системы и программирование

Магазин Бегемотик

(наименование предприятия, на котором проходила практика)

Ермакова Ирина Станиславовна, директор

(Ф.И.О. должность руководителя практики от предприятия)

Гладышева Алёна Владимировна

(Ф.И.О. руководителя практики от учебного заведения)

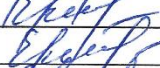
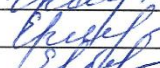
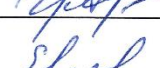
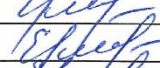
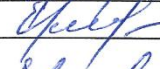
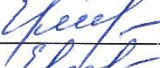
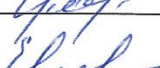
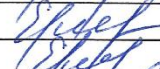
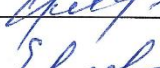
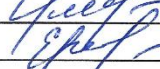

подпись ФИО руководителя практики от предприятия

ЕРМАКОВА	Подписано цифровой
ИРИНА	подписью: ЕРМАКОВА
СТАНИСЛАВОВ	ИРИНА
НА	СТАНИСЛАВОВНА
	Дата: 2026.05.22
	11:41:28 +09'00'

Чита 2026

Виды работ преддипломной практики

Включает описание проводимых работ ежедневно или еженедельно
(если виды работ не меняются):

Даты (период работ)	Выполняемые виды работ	Подпись руководителя практики от предприятия
20.04.2026	Инструктаж по технике безопасности. Ознакомление с программой практики.	
21.04.2026	Изучение требований к мобильному приложению.	
22.04.2026	Формулирование требований к приложению SportTrackler.	
23.04.2026	Проектирование архитектуры приложения.	
24.04.2026	Проектирование структуры базы данных.	
25.04.2026	Обоснование выбора средств разработки.	
26.04.2026	Настройка среды разработки. Создание каркаса приложения и базовой навигации.	
27.04.2026	Реализация модуля аутентификации.	
28.04.2026	Разработка экранов дашборда клиента и тренера.	
29.04.2026	Реализация модуля дневника питания.	
30.04.2026	Реализация модуля замеров тела с построением графиков динамики.	
01.05.2026	Разработка модуля тренировок.	
02.05.2026	Реализация системы целей.	
03.05.2026	Реализация модуля фото прогресса.	
04.05.2026	Разработка чата между клиентом и тренером.	
05.05.2026	Реализация групповых чатов с поддержкой медиафайлов и видеозаметок.	
06.05.2026	Интеграция push-уведомлений и шагомера	
07.05.2026	Реализация привязки тренера через QR-код и ввод кода вручную.	
08.05.2026	Разработка экрана настроек.	
09.05.2026	Тестирование и отладка модулей аутентификации и навигации.	
10.05.2026	Тестирование модулей питания, замеров и тренировок. Исправление ошибок.	
11.05.2026	Подготовка раздела отчёта «Программирование».	
12.05.2026	Подготовка раздела отчёта «Тестирование».	
13.05.2026	Оформление отчёта по практике.	
14.05.2026	Согласование отчёта с руководителем от предприятия.	
15.05.2026	Завершение оформления отчётной документации. Подписание отчёта.	

Всего за период практики с «20» «апреля» 2026г. по «16» «мая» 2026г.
Отработано 144 часа.

Руководитель практики

  (подпись, расшифровка подписи).

**ЕРМАКОВА
ИРИНА
СТАНИСЛАВОВ
НА**

Подписано цифровой
подписью: ЕРМАКОВА
ИРИНА
СТАНИСЛАВОВНА
Дата: 2026.05.22
11:41:28 +09'00'

« 15 мая » 2026г.